Reinforcement Learning
Lecture VII: Monte Carlo Methods

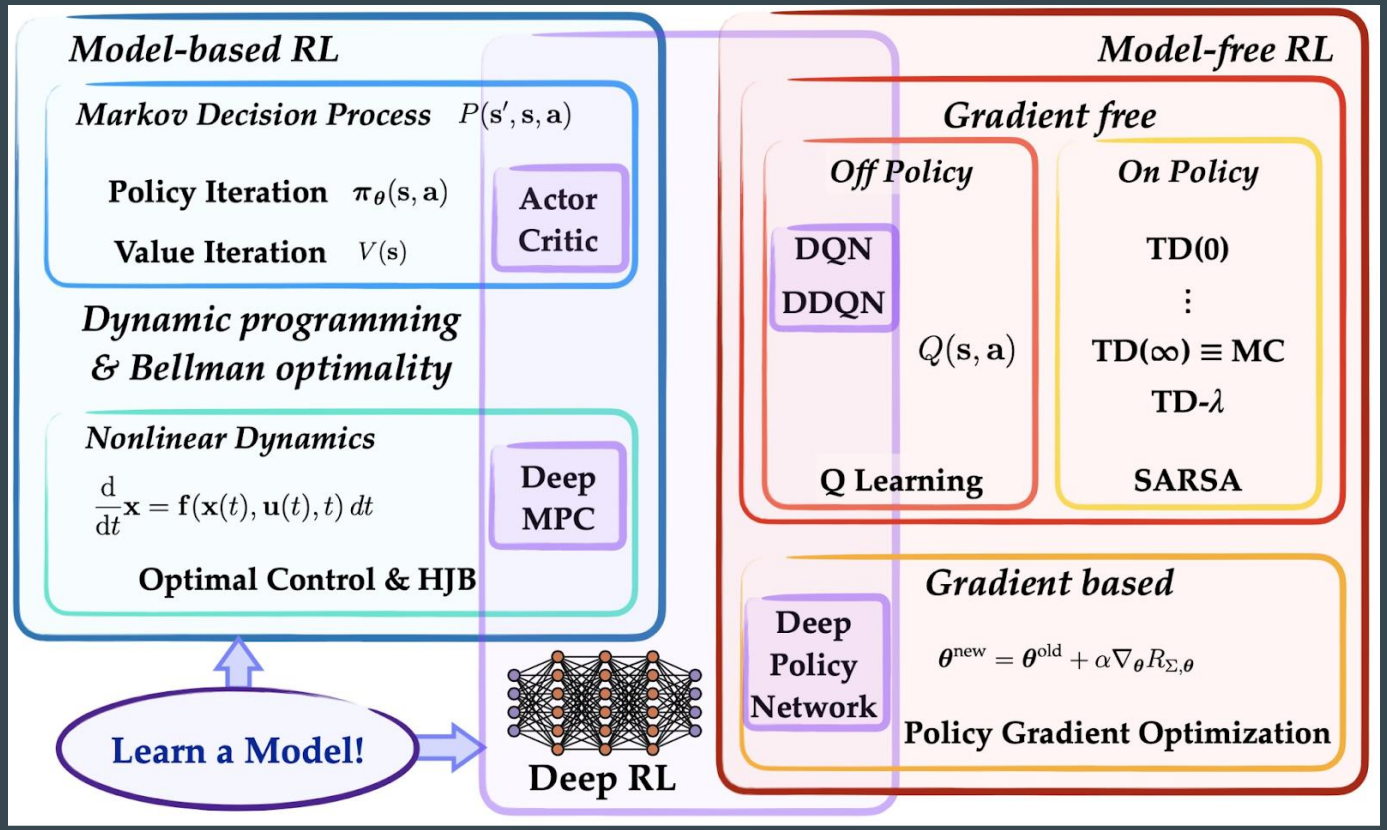
Asad Jeewa
University of Cape Town



What will be covered

- Model-free RL
- Monte Carlo Methods
- Monte Carlo Control

The Bigger Picture



Will we always have a model?

- In many real-world problems, we do not know the environmental model.
  - Model is unknown or too complex
- Without the model, how can the agent learn values?
  - Learn from experience
  - Interact with the environment
  - Observe rewards and transitions

Do tabular methods scale?

- Many real-world problems have huge or continuous state spaces.
  - Number of states grows exponentially with the number of variables
- Classic tabular methods (DP, value iteration) become intractable.

Recall: Model-free RL

- Consider: MDP $(S, A, \boldsymbol{\gamma})$
  - Transition dynamics T and reward function R are unknown
- Goal: Find an optimal policy: π^*
- How: Learn from interaction with the **environment** (experience)
  - **Monte Carlo methods**
  - Temporal Difference learning

What is Monte Carlo

- Any method that uses random sampling to estimate quantities that are difficult to compute exactly.

Method	Sampling
Monte Carlo RL	Learn value functions from episodes
Monte Carlo Search	Evaluate actions via simulated rollouts

Bootstrapping in RL

- Estimate the value of a state using other value estimates
- Example: $V_{k+1}(s)$ depends on $V_k(s')$ → using your current value function to improve itself.
- Monte Carlo waits until the end of the episode to compute the actual return G_t

MC Properties

- Model-Free Algorithm: the transition dynamics T and reward function R are unknown
- Wait until the end of an episode to update the value estimate i.e. they do not bootstrap
- Convergence may require many episodes, especially for large state spaces.
- Performance depends on variance of returns (across episodes) and exploration strategy.

First Visit MC

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

https://medium.com/@numsmt2/reinforcement-learning-chapter-5-monte-carlo-methods-part-1-monte-carlo-prediction-fcc60c9ab726

MC Prediction

- Estimating $V^{\pi}(s)$ for a fixed policy (policy is not improving).
- Is the algorithm shown synchronous or asynchronous? Why?
- Are we using the Bellman equation?

First-visit vs Every-visit

- Collect a number of episodes by following policy π.
- For each state s visited in an episode, calculate the actual return G_t that followed that visit.
- First-visit MC
  - Only the return following the first occurrence of a state s within a single episode is used to update $V(s)$.
- Every-visit MC
  - The return following every occurrence of a state s within a single episode is used to update $V(s)$.

First-visit vs Every-visit

- First-visit MC
  - Simpler
  - Unbiased estimate of V
- Every-visit MC
  - Useful if you have limited data
  - More complex because repeated state visits are correlated
  - Significant data redundancy

Analysis

- MC is the one of the simplest forms of policy evaluation based on experience.
- Advantages
  - Unbiased estimates of V^{π}: the expected value of the sample return is the true value.
  - MC does not bootstrap (like DP).
  - No need for environment model.
- Disadvantages
  - High variance: Returned observations vary significantly.
  - Classic Monte Carlo requires episodes to terminate.

From Prediction to Control

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
$Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
$Returns(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Loop forever (for each episode):

Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0
Generate an episode from S_0, A_0, following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
$G \leftarrow 0$
Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
$G \leftarrow \gamma G + R_{t+1}$
Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:
Append G to $Returns(S_t, A_t)$
$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
$\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$

https://medium.com/@numsmt2/reinforcement-learning-chapter-5-monte-carlo-methods-part-2-monte-carlo-control-b1ea0d4ec2b4

From Prediction to Control

- MC Prediction: estimate $V^{\pi}(s)$ for a fixed policy.
- Control: learn $Q(s, a)$ and improve the policy.
  - Generate episodes following an exploratory policy (ϵ-greedy). Why?
  - Update $Q(s, a)$ from sampled returns. Why?
  - Improve the policy toward greediness with respect to Q.
  - Repeat until policy converges.

Resources

- Shared RL code space
- Sutton & Barto (1998), Reinforcement Learning: An Introduction, Chapter 5.